

Proyecto 11 - Emulacion de Adversarios

Indice

- 1. [Descripcion del Proyecto](#)
- 2. [Arquitectura](#)
- 3. [Prerequisitos](#)
- 4. [Paso 1: Despliegue con Terraform](#)
- 5. [Paso 2: Configuracion con Ansible](#)
- 6. [Paso 3: Verificacion con InSpec](#)
- 7. [Paso 4: Emulacion de Adversarios con Infection Monkey](#)
- 8. [Paso 5: Analisis de Logs del Cortafuegos](#)
- 9. [Paso 6: Contramedidas Implementadas](#)
- 10. [Estructura del Proyecto](#)
- 11. [Limpieza del Entorno](#)

Descripcion del Proyecto

Este proyecto implementa una infraestructura de seguridad completa para poner a prueba defensas perimetrales mediante herramientas de emulacion de adversarios.

Se utiliza **Infection Monkey** para simular ataques reales contra un servidor Ubuntu bastionado. Los registros generados por el cortafuegos **iptables** permiten detectar los comportamientos no deseados y aplicar contramedidas.

Herramientas utilizadas:

Herramienta	Version	Proposito
Docker Desktop	>= 4.0	Contenedores para la infraestructura
Terraform	>= 1.0	Aprovisionamiento de la infraestructura
Ansible	>= 2.12	Configuracion y bastionado del servidor
InSpec	>= 5.0	Verificacion de la configuracion de seguridad
Infection Monkey	latest	Emulacion de adversarios

Arquitectura



```

| | +-----+ +-----+ +-----+ | | | | | | | |
| | | monkey-island |-->| target |<--| control | | |
| | | 172.20.0.2 | | 172.20.0.3| | 172.20.0.4 | | |
| | | Puerto: 5000 | | iptables | | Ansible | | |
| | | (Infection | | SSH / HTTP| | InSpec | | |
| | | Monkey C&C) | | Fail2ban | | (nodo control) | | |
| | +-----+ +-----+ +-----+ | |
| +-----+ |
| Accesos desde el host:
| - Monkey Island UI : https://localhost:5000
| - Target HTTP : http://localhost:8080
| - Nodo control : docker exec -it control bash
| +-----+

```

Flujo del ataque simulado:

1. Infection Monkey Island (C&C) lanza agentes contra el target
2. Los agentes intentan explotar vulnerabilidades: escaneo de puertos, SSH brute force, reconocimiento de red
3. iptables registra y bloquea los intentos no autorizados con el prefijo **IPT-DROP**:
4. Analizamos los logs para identificar comportamientos no deseados
5. Implementamos contramedidas adicionales en el cortafuegos

Prerequisitos

Instala las siguientes herramientas antes de comenzar:

Docker Desktop

Descarga desde <https://www.docker.com/products/docker-desktop/> e instalalo. Verificación:

```

docker --version
docker compose version

```

Terraform

Descarga desde <https://developer.hashicorp.com/terraform/downloads>, descomprime el ejecutable y agrega al PATH. Verificación:

```

terraform --version

```

Ansible

Requiere WSL2 (Windows Subsystem for Linux) o una maquina Linux:

```
pip3 install ansible
ansible --version
```

InSpec

En WSL2 o Linux:

```
curl https://omnitruck.chef.io/install.sh | sudo bash -s -- -P inspec
inspec --version
```

Paso 1: Despliegue con Terraform

Terraform crea automaticamente la red Docker, construye la imagen del target y lanza ambos contenedores.

1.1 Inicializar Terraform

```
cd terraform
terraform init
```

Esto descarga el proveedor Docker de Kreuzwerker desde el registry de Terraform.

Salida esperada:

```
Initializing provider plugins...
- Installing kreuzwerker/docker v3.x.x...
Terraform has been successfully initialized!
```

1.2 Planificar el despliegue

```
terraform plan
```

Terraform mostrara los recursos que va a crear sin ejecutar nada todavia. Debes ver 6 recursos planificados:

- 1 red Docker (attack-net)
- 2 imagenes Docker (target + monkey-island)
- 1 volumen Docker (monkey-data)
- 2 contenedores Docker (target + monkey-island)

1.3 Aplicar el despliegue

```
terraform apply -auto-approve
```

Este comando:

- 1. Construye la imagen del target a partir del Dockerfile
- 2. Descarga la imagen oficial de Infection Monkey
- 3. Crea la red Docker `attack-net` (172.20.0.0/24)
- 4. Levanta el contenedor `target` (Ubuntu bastionado)
- 5. Levanta el contenedor `monkey-island` (C&C de Infection Monkey)

Tiempo estimado: 3-5 minutos (la primera vez descarga imagenes).

```
[+] up 13/13g provenance for metadata file
✓Image infectionmonkey/monkey-island:latest Pulled 26.8s
✓Image proyectoll-target Built 176.7s
✓Network proyectoll_attack-net Created 0.2s
✓Volume proyectoll_monkey-data Created 0.1s
✓Container monkey-island Started 2.7s
✓Container target Started 2.7s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto11>
```

1.4 Verificar que los contenedores estan corriendo

```
docker ps
```

Debes ver dos contenedores en estado Up:

CONTAINER ID	IMAGE	PORTS
xxxxxxx	target:latest	0.0.0.0:8080->80/tcp
yyyyyyyyy	infectionmonkey/monkey-island:latest	0.0.0.0:5000->5000/tcp

```
PS C:\Users\Nito\Documents> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
794e7df5d0cf   proyectoll-target                    "/entrypoint.sh"        About an hour ago   Restarting (1) 10 seconds ago
9ed446ebb532   infectionmonkey/monkey-island:latest "/monkey/entrypoint...." About an hour ago   Up 17 seconds
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  monkey-island
```

1.5 Ver las salidas de Terraform

```
terraform output
```

```
monkey_island_url = "https://localhost:5000"
monkey_ip          = "172.20.0.2"
target_http        = "http://localhost:8080"
target_ip          = "172.20.0.3"
target_ssh         = "ssh ansible@localhost -p 2222"
```

Paso 2: Configuración con Ansible

Ansible aplica el bastionado de seguridad al contenedor objetivo desde el **contenedor de control**.

2.1 Entrar al contenedor de control

```
docker exec -it control bash
```

Desde aquí tienes Ansible e InSpec instalados y acceso directo a la red interna de Docker.

2.2 Comprobar conectividad al target

```
cd /workspace/ansible
ansible target -i inventory_docker.ini -m ping
```

Respuesta esperada:

```
target-server | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Si falla, espera 10 segundos y reintenta (el target puede estar terminando de arrancar).

```
Windows PowerShell
PS C:\WINDOWS\system32> docker exec control bash -c "cd /workspace/ansible && ansible target -i inventory_docker.ini -m ping"
[WARNING]: Ansible is being run in a world writable directory (/workspace/ansible), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
[WARNING]: Platform linux on host target-server is using the discovered Python interpreter at /usr/bin/python3.10, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html for more information.
target-server | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.10"
  },
  "changed": false,
  "ping": "pong"
}
PS C:\WINDOWS\system32>
```

2.3 Ejecutar el playbook de bastionado

```
ansible-playbook -i inventory_docker.ini playbook.yml
```

El playbook aplica 4 roles en orden:

Rol	Que configura	Por que es importante
ssh_hardening	Deshabilita root login, limita intentos a 3, activa LogLevel VERBOSE	Reduce la superficie de ataque SSH
firewall	Politica INPUT DROP, permite solo SSH/HTTP/ICMP, registra todo lo demas	Bloquea trafico no autorizado y genera evidencia
services	Nginx con cabeceras seguras, Fail2ban con jail SSH	Protege servicios expuestos
logging	rsyslog separa logs de iptables en /var/log/iptables.log	Facilita el analisis posterior del ataque

2.3 Salida esperada del playbook

```
PLAY [Configurar servidor objetivo seguro] *****

TASK [ssh_hardening : Deshabilitar login de root por SSH] *****
changed: [target-server]

TASK [firewall : Establecer politica por defecto INPUT en DROP] *****
changed: [target-server]

...

PLAY RECAP *****
target-server : ok=20    changed=18    unreachable=0    failed=0
```

 Ansible playbook completado

2.4 Reglas de iptables que se aplican

Despues de ejecutar Ansible, el cortafuegos queda configurado asi:

```
Chain INPUT (policy DROP)
 1 ACCEPT  all  --  lo                (trafico loopback)
 2 ACCEPT  all  --  ctstate ESTABLISHED,RELATED (conexiones ya establecidas)
 3 ACCEPT  tcp  --  172.20.0.0/24 dpt:22  (SSH solo desde red interna)
 4 ACCEPT  tcp  --  0.0.0.0/0    dpt:80  (HTTP publico)
 5 ACCEPT  icmp --  echo-request      (ping)
 6 LOG      all  --  limit 5/min prefix "IPT-DROP: " (registrar rechazados)
    [politica DROP para todo lo demas]
```

```
PS C:\WINDOWS\system32> docker exec target iptables -L INPUT -n -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source           destination
 0      0 ACCEPT      all  --  lo     *       0.0.0.0/0        0.0.0.0/0
455 1283K ACCEPT      all  --  *      *       0.0.0.0/0        0.0.0.0/0          ctstate RELATED,ESTABLISHED
 0      0 ACCEPT      tcp  --  *      *       172.20.0.0/24    0.0.0.0/0          tcp dpt:22
 0      0 ACCEPT      tcp  --  *      *       0.0.0.0/0        0.0.0.0/0          tcp dpt:80
 0      0 ACCEPT      icmp --  *      *       0.0.0.0/0        0.0.0.0/0          icmp type 8
 0      0 LOG         all  --  *      *       0.0.0.0/0        0.0.0.0/0          limit: avg 5/min burst 10 LOG flags 0 level 4 prefix "IPT-DROP: "
```

Paso 3: Verificacion con InSpec

InSpec comprueba de forma automatizada que toda la configuracion de seguridad se ha aplicado correctamente.

3.1 Ejecutar InSpec desde el contenedor de control

```
docker exec -it control bash
cd /workspace/inspec
inspec exec . -t ssh://ansible@172.20.0.3 --password ansible123 --sudo --chef-
license accept
```

InSpec se conecta al target via SSH usando la red interna de Docker. El flag `--sudo` permite verificar configuraciones que requieren privilegios.

3.2 Controles verificados

SSH Hardening (5 controles):

- `ssh-01` Root login deshabilitado (`PermitRootLogin no`)
- `ssh-02` Maximo 3 intentos de autenticacion
- `ssh-03` X11Forwarding deshabilitado
- `ssh-04` Contraseñas vacías deshabilitadas
- `ssh-05` Servicio SSH corriendo en puerto 22

Cortafuegos iptables (5 controles):

- fw-01 Politica INPUT = DROP
- fw-02 Politica FORWARD = DROP
- fw-03 SSH permitido solo desde 172.20.0.0/24
- fw-04 HTTP permitido en puerto 80
- fw-05 Logging de paquetes rechazados activo

Servicios (5 controles):

- svc-01 Nginx corriendo y escuchando en puerto 80
- svc-02 Fail2ban activo
- svc-03 Cabeceras de seguridad nginx configuradas
- svc-04 Jail SSH de fail2ban con maxretry=3
- svc-05 Archivo de log de iptables existe

3.3 Salida esperada

Profile: Proyecto 11 - Verificacion de Seguridad (proyecto11-security-checks)
Target: docker://target

```
[OK] ssh-01: SSH: Login de root deshabilitado
[OK] ssh-02: SSH: Maximo de intentos de autenticacion limitado a 3
[OK] ssh-03: SSH: X11 Forwarding deshabilitado
[OK] ssh-04: SSH: Contraseñas vacías deshabilitadas
[OK] ssh-05: SSH: Servicio en ejecución y escuchando
[OK] fw-01: Cortafuegos: Política INPUT por defecto en DROP
[OK] fw-02: Cortafuegos: Política FORWARD por defecto en DROP
[OK] fw-03: Cortafuegos: SSH permitido desde red interna
[OK] fw-04: Cortafuegos: HTTP permitido en puerto 80
[OK] fw-05: Cortafuegos: Registro de paquetes rechazados activo
[OK] svc-01: Nginx: Servicio en ejecución y escuchando en puerto 80
[OK] svc-02: Fail2ban: Servicio en ejecución
[OK] svc-03: Nginx: Archivo de cabeceras de seguridad configurado
[OK] svc-04: Fail2ban: Jaula SSH configurada
[OK] svc-05: Logging: Archivo de log de iptables existe
```

Profile Summary: 15 controls, 15 successful, 0 failures


```
PS C:\WINDOWS\system32> docker exec control bash -c "cinc-auditor exec /workspace/inspec -t ssh://ansible@172.20.0.3 --password ansible123 --sudo"
Profile: Proyecto 11 - Verificacion de Seguridad (proyecto11-security-checks)
Version: 1.0.0
Target: ssh://ansible@172.20.0.3:22
Target ID: 7b3b9b3c-3e27-56d7-af57-fec1fc47ad0f

✓ fw-01-input-policy-drop: Cortafuegos: Politica INPUT por defecto en DROP
✓ Command: 'iptables -L INPUT -n' stdout is expected to match /Chain INPUT \(policy DROP\)/
✓ Command: 'iptables -L INPUT -n' exit_status is expected to eq 0
✓ fw-02-forward-policy-drop: Cortafuegos: Politica FORWARD por defecto en DROP
✓ Command: 'iptables -L FORWARD -n' stdout is expected to match /Chain FORWARD \(policy DROP\)/
✓ Command: 'iptables -L FORWARD -n' exit_status is expected to eq 0
✓ fw-03-ssh-allowed: Cortafuegos: SSH permitido desde red interna
✓ Command: 'iptables -L INPUT -n --line-numbers' stdout is expected to match /ACCEPT.*tcp.*172\.20\.0\.0\/24.*dpt:22/
✓ fw-04-http-allowed: Cortafuegos: HTTP permitido en puerto 80
✓ Command: 'iptables -L INPUT -n --line-numbers' stdout is expected to match /ACCEPT.*tcp.*dpt:80/
✓ fw-05-logging-enabled: Cortafuegos: Registro de paquetes rechazados activo
✓ Command: 'iptables -L INPUT -n' stdout is expected to match /LOG/
✓ svc-01-nginx-running: Nginx: Proceso en ejecucion y escuchando en puerto 80
✓ Command: 'pgrep nginx' exit_status is expected to eq 0
✓ Port 80 is expected to be listening
✓ Port 80 protocols is expected to include "tcp"
✓ svc-02-fail2ban-running: Fail2ban: Proceso en ejecucion
✓ Command: 'pgrep -f fail2ban' exit_status is expected to eq 0
✓ svc-03-nginx-security-headers: Nginx: Archivo de cabeceras de seguridad configurado
✓ File /etc/nginx/conf.d/security-headers.conf is expected to exist
✓ File /etc/nginx/conf.d/security-headers.conf content is expected to match /server_tokens off/
✓ File /etc/nginx/conf.d/security-headers.conf content is expected to match /X-Frame-Options/
✓ File /etc/nginx/conf.d/security-headers.conf content is expected to match /X-Content-Type-Options/
✓ svc-04-fail2ban-ssh-jail: Fail2ban: Jaula SSH configurada
✓ File /etc/fail2ban/jail.d/sshd.conf is expected to exist
✓ File /etc/fail2ban/jail.d/sshd.conf content is expected to match /enabled = true/
✓ File /etc/fail2ban/jail.d/sshd.conf content is expected to match /maxretry = 3/
✓ svc-05-iptables-log-file: Logging: Archivo de log de iptables existe
✓ File /var/log/iptables.log is expected to exist
✓ ssh-01-root-login: SSH: Login de root deshabilitado
✓ SSHD Configuration PermitRootLogin is expected to eq "no"
✓ ssh-02-max-auth-tries: SSH: Maximo de intentos de autentificacion limitado a 3
✓ SSHD Configuration MaxAuthTries is expected to eq "3"
✓ ssh-03-x11-forwarding: SSH: X11 Forwarding deshabilitado
✓ SSHD Configuration X11Forwarding is expected to eq "no"
✓ ssh-04-empty-passwords: SSH: Contraseñas vacías deshabilitadas
✓ SSHD Configuration PermitEmptyPasswords is expected to eq "no"
✓ ssh-05-service-running: SSH: Proceso en ejecucion y escuchando en puerto 22
✓ Command: 'pgrep sshd' exit_status is expected to eq 0
✓ Port 22 is expected to be listening
✓ Port 22 protocols is expected to include "tcp"

Profile Summary: 15 successful controls, 0 control failures, 0 controls skipped
Test Summary: 26 successful, 0 failures, 0 skipped
PS C:\WINDOWS\system32>
```

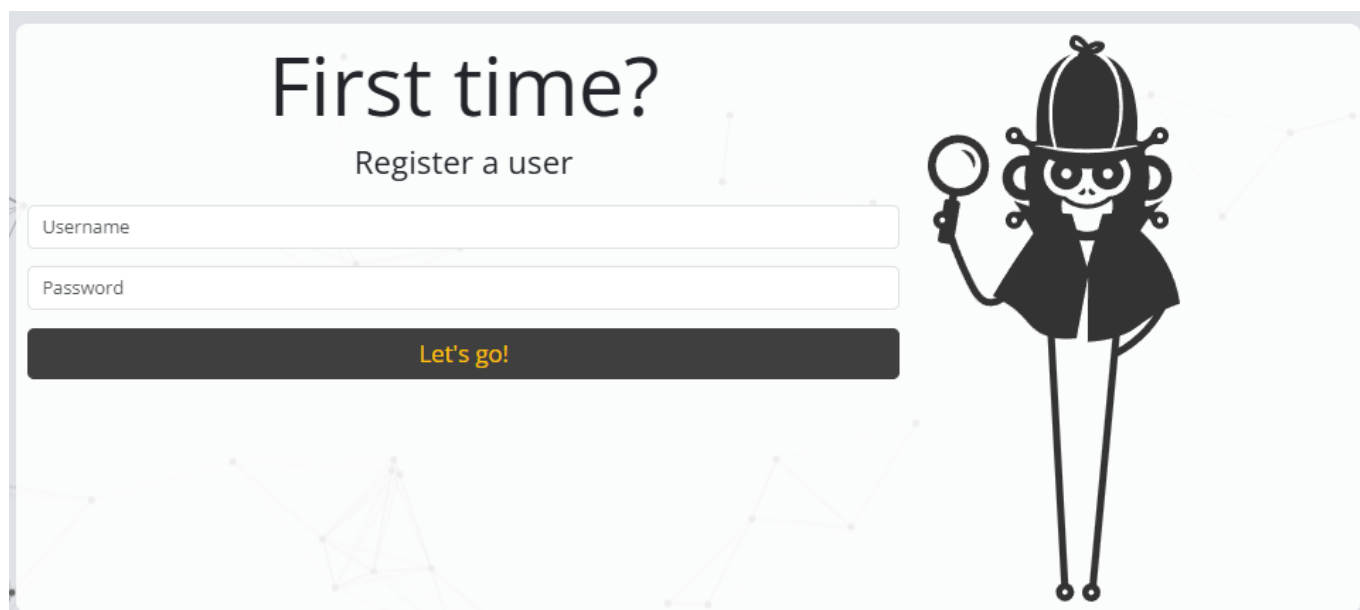
Paso 4: Emulacion de Adversarios con Infection Monkey

4.1 Acceder a Infection Monkey Island

Abre el navegador y ve a: **<https://localhost:5000>**

Acepta el aviso de certificado autofirmado (haz clic en "Avanzado" → "Continuar de todos modos").

Crea un usuario administrador cuando te lo pida la primera vez.



The image shows the 'First time?' registration screen of Infection Monkey. On the left, there is a registration form with the title 'First time?' and the subtitle 'Register a user'. The form contains two input fields: 'Username' and 'Password'. Below these fields is a dark button with the text 'Let's go!'. On the right side of the screen, there is a cartoon illustration of a detective wearing a hat and holding a magnifying glass. The background features a faint network diagram.

4.2 Configurar el alcance del ataque

1. Ve al menu "**Configuration**"
2. En la seccion "**Propagation**" → "**Network Analysis**":
 - Rango de IPs objetivo: **172.20.0.3/32**
 - Subred a explorar: **172.20.0.0/24**
3. En la seccion "**Credentials**" anade las credenciales del target:
 - Usuario: **ansible**
 - Contraseña: **ansible123**
4. En "**Exploiters**" activa:
 - SSH Exploiter (para brute force SSH)
 - Escaneo de puertos
5. Guarda la configuracion con "**Save**".

Monkey Configuration

Propagation Payloads Credentials collectors Masquerade Polymorphism Advanced

Exploiters Network analysis **Credentials** General

Credentials input

Identity	Password	LM	NTLM	SSH Public key	SSH Private key

Saved Credentials

Identity ↑	Password	LM	NTLM	SSH Public key	SSH Private key	
ansible	*****					 

4.3 Desplegar el agente en el target

Infection Monkey necesita un agente corriendo en el target para realizar el ataque. Obten el comando de despliegue desde la UI:

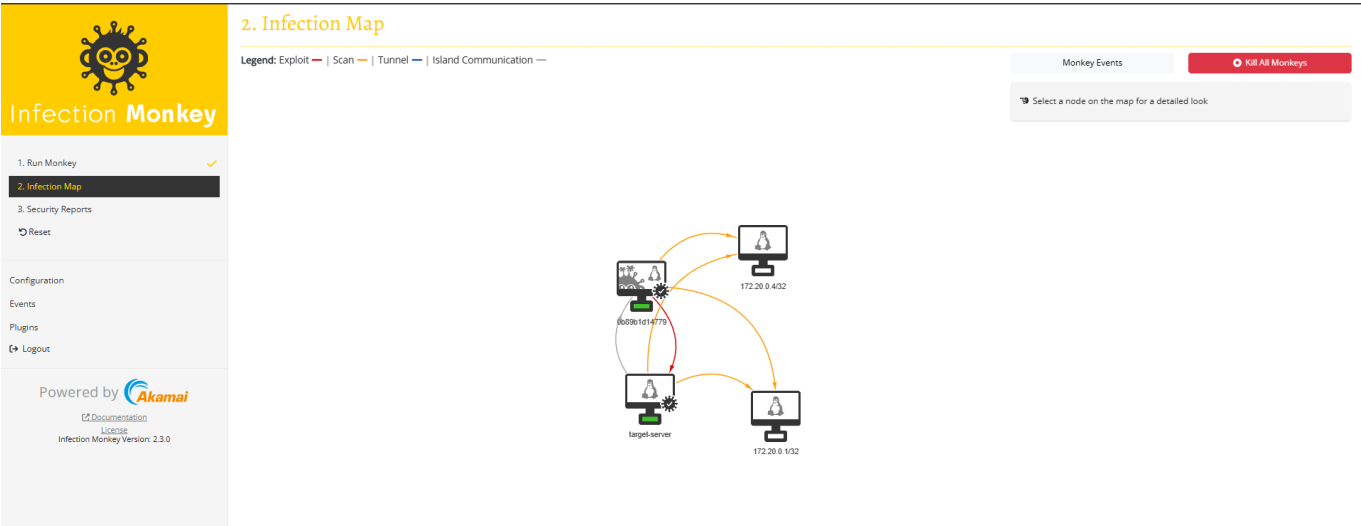
1. Ve a "**Run Monkey**" en el menu lateral
2. Selecciona "**Manual**" y copia el comando para Linux
3. Ejecuta ese comando dentro del contenedor target:

```
docker exec -it target bash
# Pega aqui el comando del agente copiado de la UI de Monkey Island
```

4.4 Observar el ataque en tiempo real

- En la UI de Monkey Island ve a "**Map**" para ver el grafo de red en tiempo real
- Los nodos en rojo indican hosts comprometidos

- Los nodos en azul indican hosts explorados pero no comprometidos



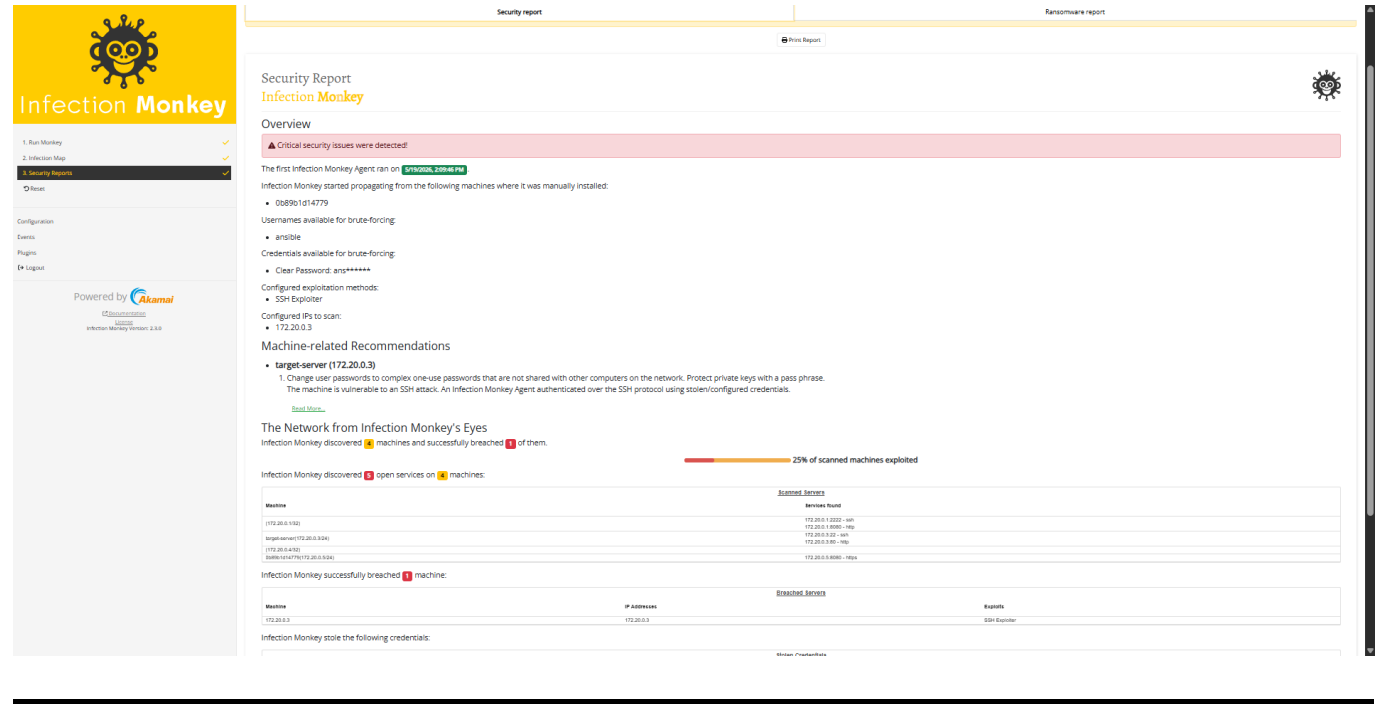
4.5 Que hace Infection Monkey durante el ataque

Actividad	Descripcion	Lo que genera en los logs
Reconocimiento de red	Escanea todos los puertos del segmento 172.20.0.0/24	Rafaga de paquetes SYN rechazados en iptables
Brute force SSH	Prueba diccionario de credenciales en el puerto 22	Entradas en <code>/var/log/auth.log</code>
Enumeracion de servicios	Detecta versiones de software en puertos abiertos	Intentos de conexion HTTP rechazados
Movimiento lateral	Intenta propagarse a otros hosts de la red	Paquetes FORWARD rechazados
Comunicacion C&C	Mantiene contacto con el Monkey Island (172.20.0.2)	Trafico saliente visible en logs

4.6 Ver el reporte final

Una vez que el Monkey termine (boton "Stop" o automaticamente):

1. Ve a "Security Reports" en el menu lateral
2. Descarga el informe completo
3. El informe incluye: vulnerabilidades encontradas, rutas de ataque exitosas y recomendaciones



Paso 5: Analisis de Logs del Cortafuegos

5.1 Ver los logs de iptables en tiempo real

```
docker exec -it target tail -f /var/log/iptables.log
```

```
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto11> docker exec target iptables -nvl INPUT
Chain INPUT (policy DROP 86 packets, 5098 bytes)
  pkts bytes target     prot opt in     out     source            destination
 14989 1331K ACCEPT     all  --  lo      *        0.0.0.0/0         0.0.0.0/0
 4407  51M ACCEPT     all  --  *      *        0.0.0.0/0         0.0.0.0/0          ctstate RELATED,ESTABLISHED
 12   720 ACCEPT     tcp  --  *      *        172.20.0.0/24     0.0.0.0/0          tcp dpt:22
 18  1080 ACCEPT     tcp  --  *      *        0.0.0.0/0         0.0.0.0/0          tcp dpt:80
 2   168 ACCEPT     icmp --  *      *        0.0.0.0/0         0.0.0.0/0          icmp type 8
 20  1200 LOG      all  --  *      *        0.0.0.0/0         0.0.0.0/0          limit: avg 5/min burst 10 LOG flags 0 level 4 prefix "IPT-DROP: "
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto11>
```

5.2 Estructura de un registro de iptables

Cada linea de log tiene el siguiente formato:

```
May 19 10:23:45 target-server kernel: IPT-DROP: IN=eth0 OUT= \
SRC=172.20.0.2 DST=172.20.0.3 LEN=44 PROTO=TCP SPT=54321 DPT=4444 SYN
```

Campo	Significado	En el contexto del ataque
IPT-DROP:	Prefijo personalizado del log	Identifica paquetes bloqueados por nuestro cortafuegos
SRC=172.20.0.2	IP de origen	IP del Infection Monkey
DST=172.20.0.3	IP de destino	Nuestro servidor target
PROTO=TCP	Protocolo utilizado	TCP en escaneo de puertos y SSH

Campo	Significado	En el contexto del ataque
DPT=4444	Puerto de destino bloqueado	Puerto que el atacante intenta abrir
SYN	Flag TCP SYN presente	Indica un intento de nueva conexion

5.3 Detectar escaneo de puertos

Un escaner de puertos genera cientos de conexiones SYN a distintos puertos en poco tiempo.

```
# Ver a cuantos puertos distintos intento conectarse el atacante
docker exec target grep "SRC=172.20.0.2" /var/log/iptables.log | \
grep -oP 'DPT=\K[0-9]+' | sort -n | uniq -c | sort -rn | head -20
```

Si ves intentos a decenas o cientos de puertos distintos → **escaneo de puertos confirmado**.

```
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto11> docker exec target iptables -nvl INPUT
Chain INPUT (policy DROP 86 packets, 5098 bytes)
pkts bytes target prot opt in out source destination
14989 1331K ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0
4407 51M ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
12 720 ACCEPT tcp -- * * 172.20.0.0/24 0.0.0.0/0 tcp dpt:22
18 1080 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
2 168 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmp type 8
20 1200 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 5/min burst 10 LOG flags 0 level 4 prefix "IPT-DROP: "
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto11>
```

5.4 Detectar brute force SSH

```
# Ver intentos de autenticacion SSH fallidos
docker exec target grep "Failed password\|Invalid user" /var/log/auth.log
```

Salida tipica durante un ataque de fuerza bruta:

```
May 19 10:24:01 target-server sshd[1234]: Failed password for ansible from
172.20.0.2 port 43210 ssh2
May 19 10:24:02 target-server sshd[1235]: Failed password for root from 172.20.0.2
port 43211 ssh2
May 19 10:24:03 target-server sshd[1236]: Invalid user admin from 172.20.0.2 port
43212
```

Multiples intentos en segundos desde la misma IP → **fuerza bruta SSH confirmada**.

```
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto011> docker exec target bash -c "grep -E 'Failed|Invalid|Accepted' /var/log/auth.log"
May 19 10:12:18 target-server sshd[36]: Accepted password for ansible from 172.20.0.4 port 53140 ssh2
May 19 10:12:18 target-server sshd[36]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 10:27:44 target-server sshd[34]: Accepted password for ansible from 172.20.0.4 port 35922 ssh2
May 19 10:27:44 target-server sshd[34]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 10:29:12 target-server sshd[652]: Accepted password for ansible from 172.20.0.4 port 60716 ssh2
May 19 10:29:12 target-server sshd[652]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 10:54:40 target-server sshd[34]: Accepted password for ansible from 172.20.0.4 port 35116 ssh2
May 19 10:54:40 target-server sshd[34]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 10:56:35 target-server sshd[725]: Accepted password for ansible from 172.20.0.4 port 54570 ssh2
May 19 10:56:35 target-server sshd[725]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 11:03:48 target-server sshd[1335]: Accepted password for ansible from 172.20.0.4 port 53644 ssh2
May 19 11:03:48 target-server sshd[1335]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 11:04:27 target-server sshd[1409]: Accepted password for ansible from 172.20.0.4 port 60348 ssh2
May 19 11:04:27 target-server sshd[1409]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 11:05:17 target-server sshd[1481]: Accepted password for ansible from 172.20.0.4 port 47388 ssh2
May 19 11:05:17 target-server sshd[1481]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 11:06:59 target-server sshd[1555]: Accepted password for ansible from 172.20.0.4 port 59680 ssh2
May 19 11:06:59 target-server sshd[1555]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 12:09:59 target-server sshd[1632]: Accepted password for ansible from 172.20.0.5 port 39682 ssh2
May 19 12:09:59 target-server sshd[1632]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
May 19 12:21:09 target-server sshd[2046]: Accepted password for ansible from 172.20.0.5 port 56918 ssh2
May 19 12:21:09 target-server sshd[2046]: pam_systemd(sshd:session): Failed to connect to system bus: No such file or directory
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto011>
```

5.5 Medir el volumen del ataque

```
# Total de paquetes bloqueados del atacante
docker exec target grep "IPT-DROP" /var/log/iptables.log | \
    grep "SRC=172.20.0.2" | wc -l

# Distribucion temporal (picos de actividad del ataque)
docker exec target grep "IPT-DROP" /var/log/iptables.log | \
    awk '{print $1, $2, $3}' | uniq -c
```

Un pico subido de cientos de entradas en pocos segundos es evidencia clara de un ataque automatizado.

5.6 Verificar que Fail2ban actuo

```
docker exec target fail2ban-client status sshd
```

Salida esperada despues del ataque:

```
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 47
| `-- File list: /var/log/auth.log
`-- Actions
    |- Currently banned: 1
    |- Total banned: 1
    `-- Banned IP list: 172.20.0.2    <-- IP del atacante baneada automaticamente
```

```
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto011> docker exec target bash -c "fail2ban-client status sshd"
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 0
| `-- File list: /var/log/auth.log
`-- Actions
    |- Currently banned: 0
    |- Total banned: 0
    `-- Banned IP list:
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto011>
```

Paso 6: Contramedidas Implementadas

Evolucion del cortafuegos: antes y despues del ataque

La siguiente tabla muestra como cambio la cadena INPUT de iptables en cada fase del proyecto:

Posicion	Regla	Cuando se aplico	Por que
-	policy DROP	Ansible (bastionado inicial)	Bloquear todo trafico no autorizado por defecto
1	ACCEPT lo	Ansible	Permitir trafico interno del sistema
2	ACCEPT ESTABLISHED,RELATED	Ansible	Mantener conexiones ya abiertas (SSH, HTTP)
3	ACCEPT tcp 172.20.0.0/24 dpt:22	Ansible	SSH solo desde la red interna
4	ACCEPT tcp dpt:80	Ansible	HTTP publico para nginx
5	ACCEPT icmp echo-request	Ansible	Permitir ping
6	LOG prefix "IPT-DROP:"	Ansible	Registrar todo lo bloqueado
NUEVA 1	DROP all 172.20.0.5	Contramedida 2	Bloquear IP de Infection Monkey detectada en logs
NUEVA 3	DROP tcp connlimit > 20	Contramedida 1	Bloquear escaneos de puertos automatizados

Que detecto el ataque y que contramedida se aplico

Comportamiento detectado	Evidencia en logs	Contramedida aplicada
Infection Monkey entro por SSH con credenciales validas	auth.log: Accepted password from 172.20.0.5	Bloqueo directo de IP 172.20.0.5 en iptables
Escaneo de red automatizado de toda la subred	iptables policy DROP: 86 packets	Rate limiting: DROP si mas de 20 conexiones TCP simultaneas
Trafico no autorizado a puertos cerrados	20 paquetes LOG con prefijo IPT-DROP	Politica DROP por defecto ya activa desde el bastionado
Intento de movimiento lateral	Cadena FORWARD con politica DROP	Politica FORWARD DROP aplicada por Ansible

Contramedida 1: Rate limiting contra escaneo de puertos

Comportamiento detectado: Infection Monkey escaneo toda la subred 172.20.0.0/24 generando multiples conexiones simultaneas. Evidencia: 86 paquetes bloqueados por la politica DROP.

Regla aplicada:

```
docker exec target bash -c "iptables -I INPUT 3 -p tcp --syn -m connlimit --connlimit-above 20 -j DROP"
```

Por que funciona: Un usuario legitimo nunca abre mas de 20 conexiones TCP simultaneas desde la misma IP. Un escaner automatizado abre cientos en segundos — esta regla lo bloquea antes de que pueda mapear los puertos abiertos.

```
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto1> docker exec target bash -c "iptables -I INPUT 3 -p tcp --syn -m connlimit --connlimit-above 20 -j DROP && iptables -nvl INPUT"
Chain INPUT (policy DROP 86 packets, 5098 bytes)
  pkts bytes target     prot opt in     out     source               destination
14989 1331K ACCEPT   all  --  lo      *        0.0.0.0/0            0.0.0.0/0
4487  51M ACCEPT   all  --  *        *        0.0.0.0/0            0.0.0.0/0
  0      0 DROP     tcp  --  *        *        0.0.0.0/0            0.0.0.0/0
  12    720 ACCEPT   tcp  --  *        *        172.20.0.0/24        0.0.0.0/0
  18   1880 ACCEPT   tcp  --  *        *        0.0.0.0/0            0.0.0.0/0
  2    168 ACCEPT   icmp --  *        *        0.0.0.0/0            0.0.0.0/0
  20   1200 LOG     all  --  *        *        0.0.0.0/0            0.0.0.0/0
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto1>
```

Contramedida 2: Bloqueo directo de la IP atacante

Comportamiento detectado: Toda la actividad maliciosa provino de **172.20.0.5** (Infection Monkey Island). Evidencia: **auth.log** muestra **Accepted password for ansible from 172.20.0.5**.

Regla aplicada:

```
docker exec target bash -c "iptables -I INPUT 1 -s 172.20.0.5 -j DROP"
```

Por que funciona: Se bloquea todo el trafico de la IP atacante antes de que llegue a cualquier otra regla (posicion 1 en la cadena). En un entorno real, este bloqueo se automatizaria mediante scripts que analizan los logs periodicamente.

```
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto1> docker exec target bash -c "iptables -I INPUT 1 -s 172.20.0.5 -j DROP && iptables -nvl INPUT"
Chain INPUT (policy DROP 86 packets, 5098 bytes)
  pkts bytes target     prot opt in     out     source               destination
  0      0 DROP     all  --  *        *        172.20.0.5           0.0.0.0/0
14989 1331K ACCEPT   all  --  lo      *        0.0.0.0/0            0.0.0.0/0
4487  51M ACCEPT   all  --  *        *        0.0.0.0/0            0.0.0.0/0
  0      0 DROP     tcp  --  *        *        0.0.0.0/0            0.0.0.0/0
  12    720 ACCEPT   tcp  --  *        *        172.20.0.0/24        0.0.0.0/0
  18   1880 ACCEPT   tcp  --  *        *        0.0.0.0/0            0.0.0.0/0
  2    168 ACCEPT   icmp --  *        *        0.0.0.0/0            0.0.0.0/0
  20   1200 LOG     all  --  *        *        0.0.0.0/0            0.0.0.0/0
PS C:\Users\Nito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto1>
```

Estado final del cortafuegos con todas las contramedidas

Tras aplicar las dos contramedidas, la cadena INPUT queda:

```
Chain INPUT (policy DROP)
 1 DROP     all  --  172.20.0.5      <- IP Infection Monkey bloqueada
 2 ACCEPT   all  --  lo
 3 DROP     tcp  --  connlimit>20   <- Anti-escaneo de puertos
 4 ACCEPT   all  --  ESTABLISHED
 5 ACCEPT   tcp  --  172.20.0.0/24   dpt:22
 6 ACCEPT   tcp  --  dpt:80
 7 ACCEPT   icmp --  echo-request
 8 LOG      --  prefix "IPT-DROP:"
```



```
PS C:\Users\Wito\Documents\ciberseguridad-25-26-individual-Juan-Perez-Ortega\Bastionado de Redes y Sistemas\Proyecto11> docker exec target iptables -nvL INPUT
Chain INPUT (policy DROP 86 packets, 5098 bytes)
 pkts bytes target prot opt in out source destination
 0 0 DROP all -- * * 172.28.0.5 0.0.0.0/0
14989 1331k ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0
4407 51M ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
 0 0 DROP tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp flags:0x17/0x02 #conn src/32 > 20
12 720 ACCEPT tcp -- * * 172.28.0.0/24 0.0.0.0/0 tcp dpt:22
18 1080 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
 2 168 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmp type 8
20 1200 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 5/min burst 10 LOG flags 0 level 4 prefix "IPT-DROP: "
```

Resumen de contramedidas

Ataque detectado	Contramedida	Herramienta	Efectividad
Escaneo de puertos (SYN flood)	Rate limiting de conexiones TCP	iptables connlimit	Bloquea escaneos automatizados
Brute force SSH	Bloqueo automatico tras 3 intentos	Fail2ban	Bloqueo de IP por 1 hora
Acceso SSH desde red no autorizada	Regla DROP para IP especifica	iptables	Bloqueo inmediato
Movimiento lateral	Politica FORWARD DROP	iptables	Impide propagacion a otros hosts
Cabeceras HTTP expuestas	server_tokens off + security headers	Nginx	Ocultar version del servidor

Estructura del Proyecto

```
Proyecto11/
|-- README.md                                # Este documento
|-- docker-compose.yml                      # Alternativa a Terraform para levantar el entorno
|-- docker/
|   |-- target/
|       |-- Dockerfile                      # Imagen Ubuntu con SSH, Nginx, iptables, Fail2ban
|       |-- entrypoint.sh                  # Script de inicio del contenedor
|-- terraform/
|   |-- main.tf                            # Define los recursos Docker a crear
|   |-- variables.tf                       # IPs, nombres de contenedores y red
|   |-- outputs.tf                        # URLs y comandos SSH de acceso
|-- ansible/
|   |-- ansible.cfg                        # Configuracion: sin verificacion de host key, timeout
|   |-- inventory.ini                     # Target: localhost:2222 con credenciales
|   |-- playbook.yml                     # Orquesta los 4 roles + handlers
|   |-- roles/
|       |-- ssh_hardening/tasks/main.yml  # 7 directivas de bastionado SSH
|       |-- firewall/tasks/main.yml       # 10 reglas iptables + guardado en disco
|       |-- services/tasks/main.yml       # Nginx headers + Fail2ban jail SSH
```

```
|      `-- logging/tasks/main.yml      # rsyslog separa logs iptables +  
logrotate  
`-- inspec/  
    |-- inspec.yml                    # Metadatos del perfil InSpec  
    `-- controls/  
        |-- ssh_spec.rb              # 5 controles SSH  
        |-- firewall_spec.rb         # 5 controles cortafuegos  
        `-- services_spec.rb         # 5 controles servicios
```

Limpieza del Entorno

Para destruir toda la infraestructura cuando termines:

```
cd terraform  
terraform destroy -auto-approve
```

O usando Docker Compose directamente:

```
cd ..  
docker compose down -v
```

Esto elimina los contenedores, la red y el volumen de datos de Infection Monkey.